

知枢

企业知识资产管理与智能检索平台

ER 图与关系模式宣讲版

本版 PDF 用于答辩和宣讲，重点统一到 “2 张核心 ER 图 + 1 套中文关系模式”。概念层只保留真正独立的核心实体，不把实现层日志、对象存储或向量细节硬塞进 ER 图。

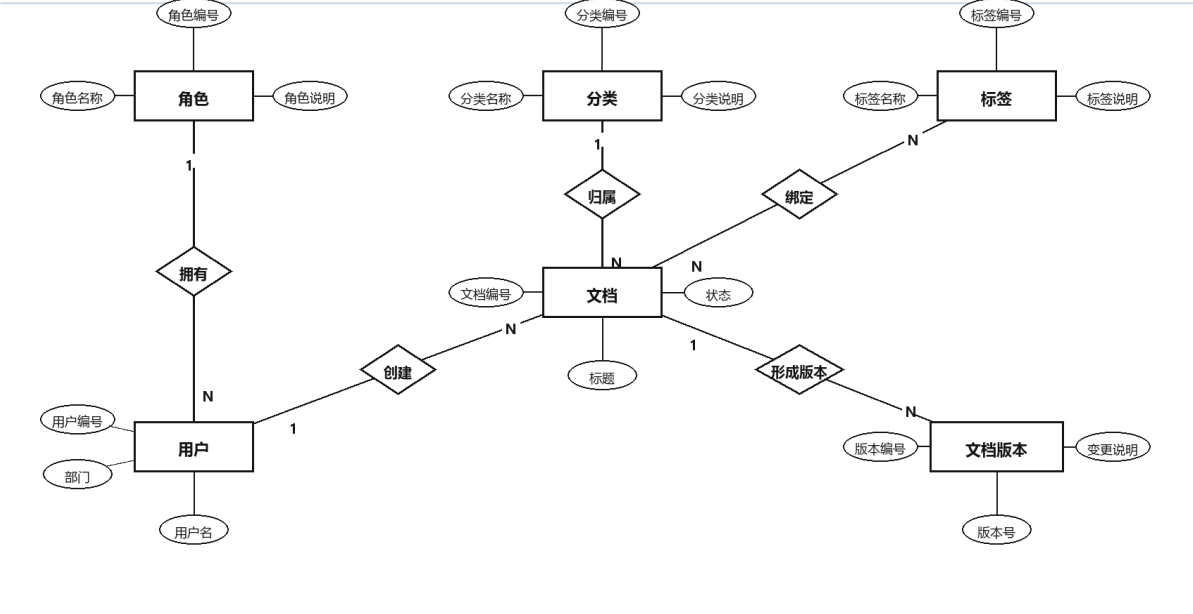
- ER-1 先讲基础知识管理域，建立知识库主干。
- ER-2 再讲问答与版本追溯域，说明答案为什么可解释。
- 关系模式只跟这两张图对应，不再把实现层细节混进概念层。

子图	覆盖实体	讲解重点
ER-1 基础知识管理域	角色 / 用户 / 分类 / 文档 / 标签 / 文档版本	谁在建文档、文档属于哪类、文档如何打标签和形成版本
ER-2 问答与版本追溯域	用户 / 问题 / 回答 / 引用证据 / 文档 / 文档版本	回答如何通过引用证据回溯到具体文档版本

ER-1 基础知识管理域

ER-1 基础知识管理域

概念层只保留真正独立的核心实体：角色、用户、分类、文档、标签、文档版本，边上标注 1 或 N。



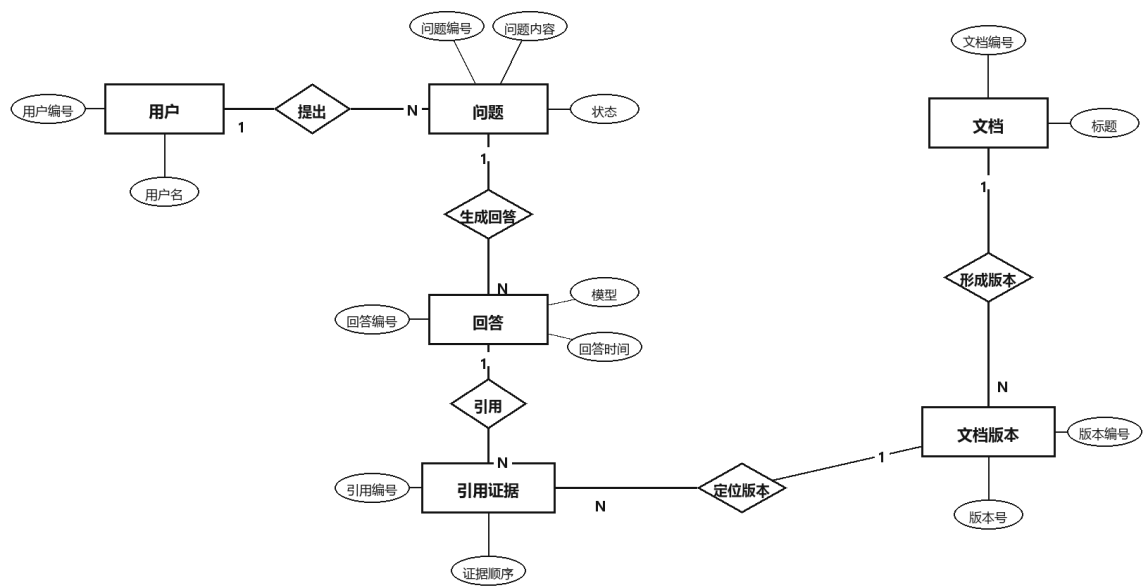
图示重点：先讲清谁在维护文档、文档属于哪类、文档如何打标签以及如何形成版本。

- 这一张图只保留最核心的知识管理实体，不再把阅读记录、收藏记录这些实现层行为画成概念实体。
- 角色到用户是 1:N，说明权限身份从角色出发；用户到文档是 1:N，说明文档有明确责任主体。
- 分类到文档是 1:N，说明分类承担稳定归档作用；文档到标签是 N:N，说明标签承担灵活标注作用。
- 文档到文档版本是 1:N，说明一个文档会形成多个历史版本，这是知识追溯的基础。

ER-2 问答与版本追溯域

ER-2 问答与版本追溯域

概念层只保留问答闭环里最关键的对象：用户、问题、回答、引用证据、文档、文档版本，边上标注 1 或 N。



图示重点：回答不是黑盒文本，它可以通过引用证据回溯到具体文档版本。

- 这一张图只保留问答闭环里真正需要讲清楚的 6 个核心对象，不再把片段、文件、Agent 运行这些实现层对象放进概念图。
- 用户到问题是 1:N，问题到回答是 1:N，表示一个用户可以提多个问题，一个问题也可能生成多次回答。
- 回答到引用证据是 1:N，这是关键设计，因为一个回答往往需要多条证据支撑。
- 引用证据再定位到文档版本，表示答案依据的是某个具体版本，而不是一篇抽象文档。

关系模式怎么讲

讲关系模式时，先把这 2 张 ER 图翻译成中文关系模式。最容易讲的规则是：一对多联系并入多的一端，多对多联系单独变成中间关系模式。

中文关系模式	说明
角色(角色编号 PK, 角色名称, 角色说明)	主角色实体
用户(用户编号 PK, 角色编号 FK, 用户名, 部门)	角色通过外键并入用户
分类(分类编号 PK, 分类名称, 分类说明)	文档归属实体
文档(文档编号 PK, 分类编号 FK, 创建者编号 FK, 标题, 状态)	分类与创建联系并入文档
标签(标签编号 PK, 标签名称, 标签说明)	灵活标注实体
文档版本(版本编号 PK, 文档编号 FK, 版本号, 变更说明)	文档形成多个版本
问题(问题编号 PK, 用户编号 FK, 问题内容, 状态)	用户提出的问题
回答(回答编号 PK, 问题编号 FK, 模型, 回答时间)	问题生成的回答
引用证据(引用编号 PK, 回答编号 FK, 版本编号 FK, 证据顺序)	把回答与版本依据连接起来
文档标签(文档编号 PK/FK, 标签编号 PK/FK)	多对多联系单独转换

核心结构	你要强调的点	一句话讲法
documents + document_versions	当前快照与历史版本分离	主表负责当前展示，版本表负责历史追溯。
answers + answer_citations	答案与证据解耦	一个回答可以对应多条证据，不再局限单文档引用。
documents + tags + document_tags	分类与灵活标签分离	分类负责稳定归档，标签负责灵活标注。
users + questions + answers	问答责任链清晰	谁提问、问题如何生成回答，都可以明确追溯。

老师常见追问的简答模板

- 为什么不把阅读记录、收藏记录画成核心实体：因为它们更像实现层行为表，不是概念层主对象。
- 为什么不把文件、片段、Agent 运行画进主 ER 图：因为它们更适合放在实现设计里，而不是课程设计的概念模型里。
- 为什么 documents 和 document_versions 是 1:N：因为一个文档会多次修订，主表负责当前态，版本表负责历史态。
- 为什么 answers 不直接连 documents：因为真实 RAG 是一答多证据，所以要单独建引用证据。

宣讲顺序建议：业务目标 → 2 张核心 ER 图 → 中文关系模式 → 3 个关键结构亮点。这样更像数据库课程设计的标准答辩，也更容易解释为什么有些东西该画成实体，有些东西只适合做属性或实现表。